Aula 29: Fundamentos do React e o Virtual DOM

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

Índice

- INTRODUÇÃO
- MANIPULAÇÃO DO DOM
- PROCESSO DE RECONCILIAÇÃO
- DIFFING EM AÇÃO
- JSX
- REGRAS DO JSX
- JSX COM CHAVES
- COMPONENTE REACT
- ANATOMIA DE UM COMPONENTE REACT
- REACT EM PRODUÇÃO
- EXERCÍCIOS

Introdução

- React é uma biblioteca JavaScript para construir interfaces de usuário (UIs)
- **Declarativo**: Você descreve como sua UI deve parecer em qualquer estado, e o React cuida de atualizá-la eficientemente quando os dados mudam
- **Baseado em Componentes**: Crie componentes encapsulados que gerenciam seu próprio estado e depois componha-os para criar UIs complexas
- **Aprenda uma vez, escreva em qualquer lugar**: O React pode ser usado para desenvolver aplicações web, mobile (com React Native) e desktop

Introdução

COMO O REACT FUNCIONA

- Cria um DOM virtual na memória
- Em vez de manipular o DOM do navegador diretamente, cria um DOM virtual na memória
- Realiza todas as manipulações necessárias antes de alterar o DOM do navegador
- Altera apenas o que precisa ser alterado
- Detecta as alterações feitas e altera somente o que precisa ser alterado.

Manipulação do DOM

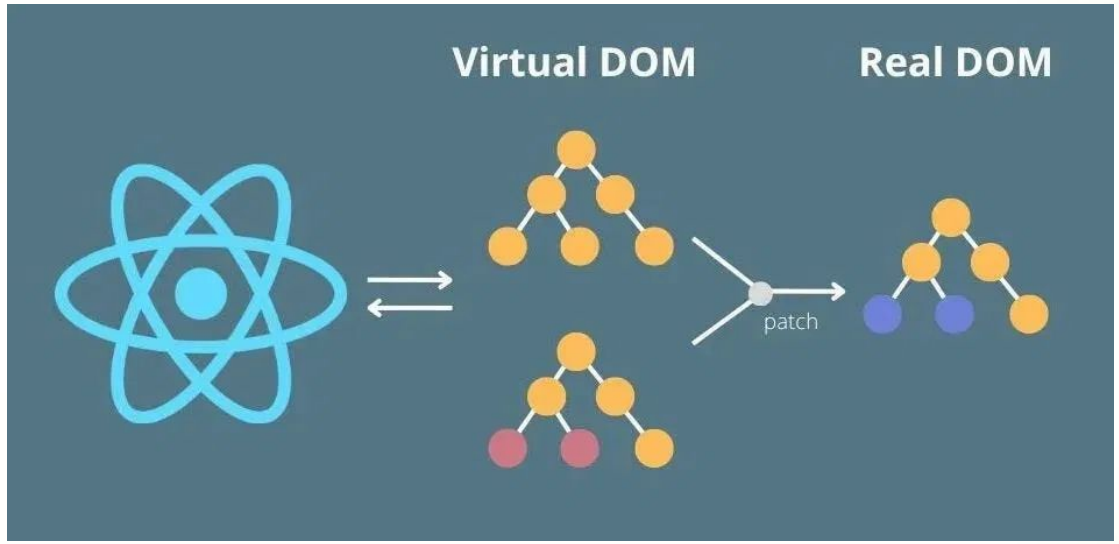
O PROBLEMA

- Manipular o DOM diretamente é lento e complexo.
- O Document Object Model (DOM) é uma representação em árvore da página web
- Alterações diretas no DOM são custosas para o navegador, pois cada atualização pode forçar o recalculamento de estilos e layout (reflow/repaint)
- Isso impacta a performance

Manipulação do DOM

SOLUÇÃO: VIRTUAL DOM

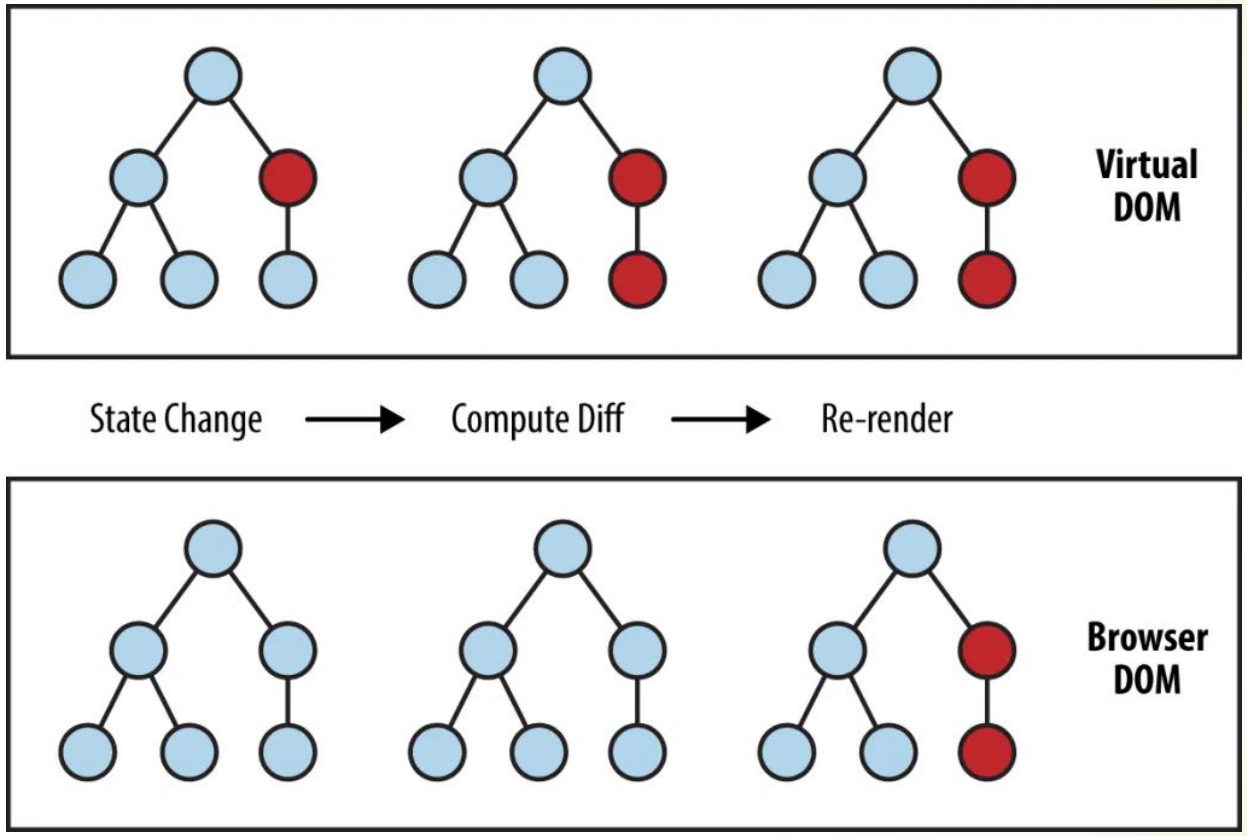
- Representação leve: Uma cópia em memória desacoplada do DOM real
- Performance: Manipulações extremamente rápidas, sem tocar na API do navegador inicialmente
- Otimização: O React agrupa as alterações e as aplica de forma eficiente



Processo de Reconciliação

ALGORITMO EFICIENTE DO REACT

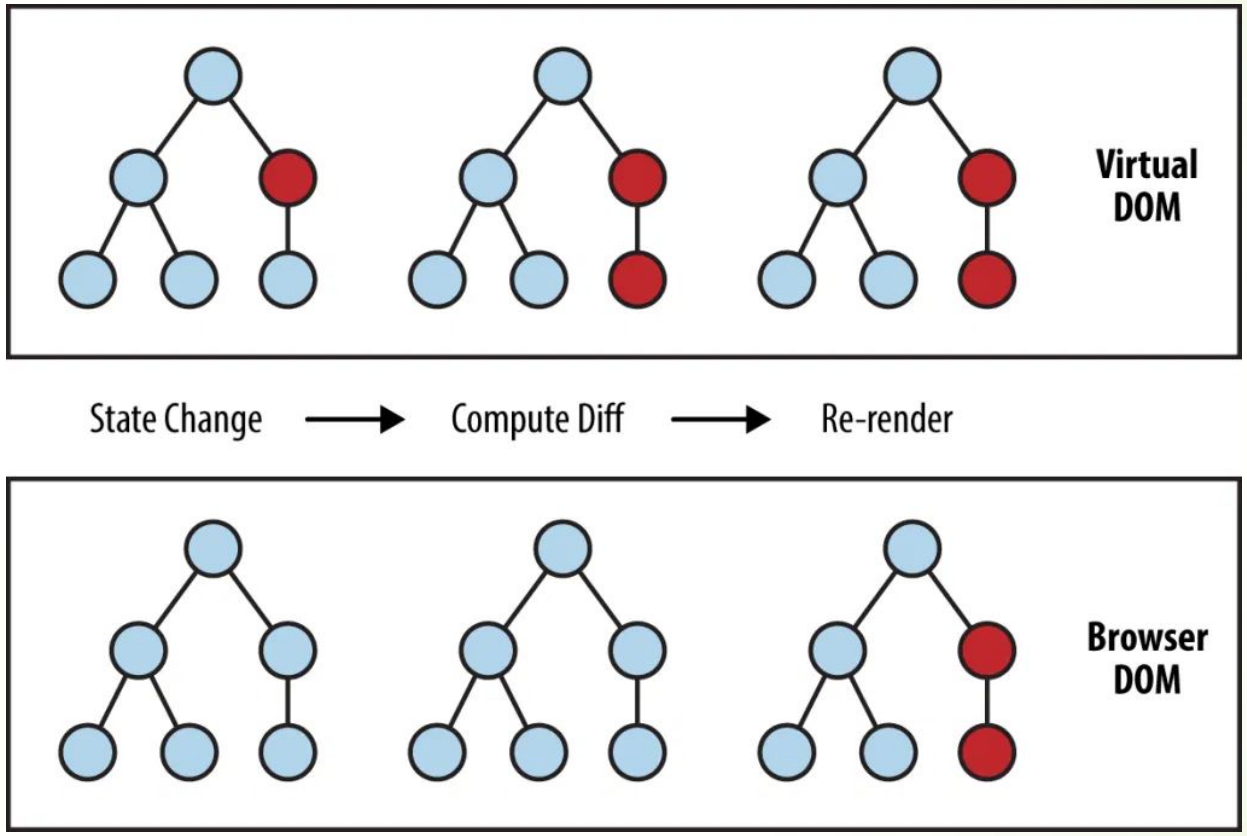
- **Mudança de Estado**: Quando o estado muda, o React cria uma nova árvore do Virtual DOM
- **Diffing**: O React compara a nova árvore com a versão anterior (snapshot)



Processo de Reconciliação

ALGORITMO EFICIENTE DO REACT

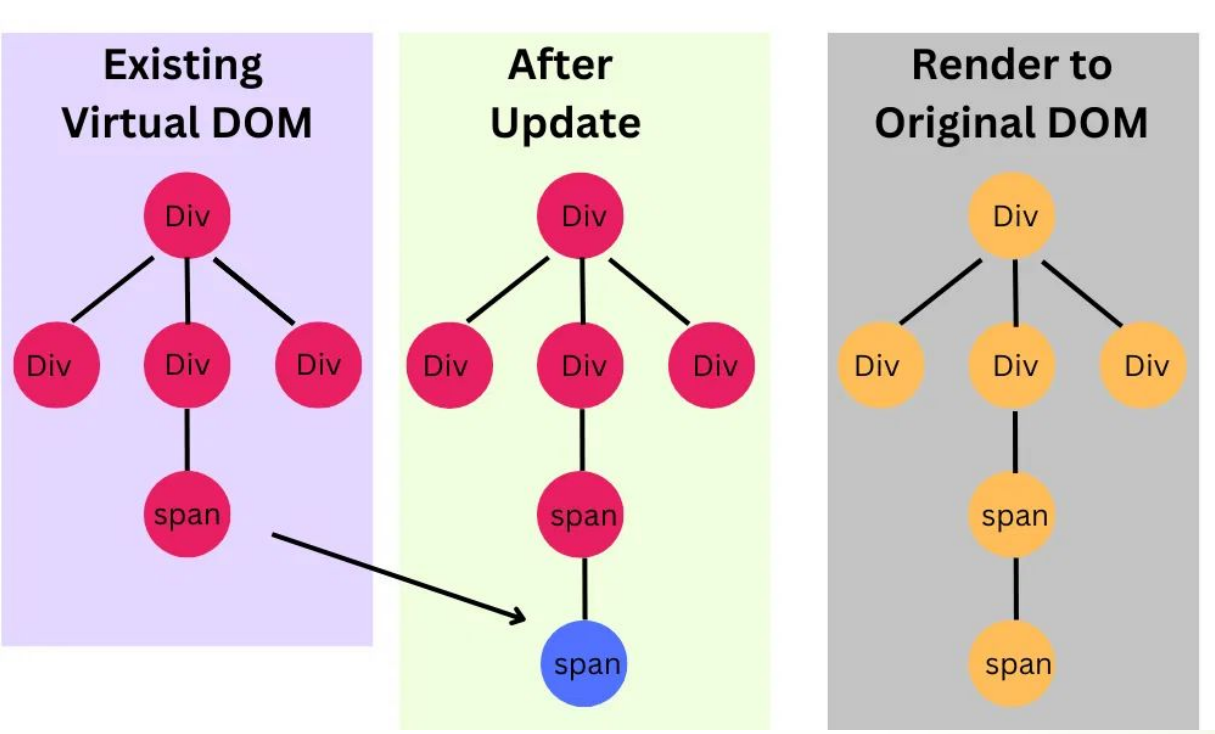
- **Atualização em Lote**: O React calcula o mínimo de mudanças e as aplica ao DOM real de uma só vez



Diffing em Ação

HEURÍSTICA PARA EFICIÊNCIA O(n)

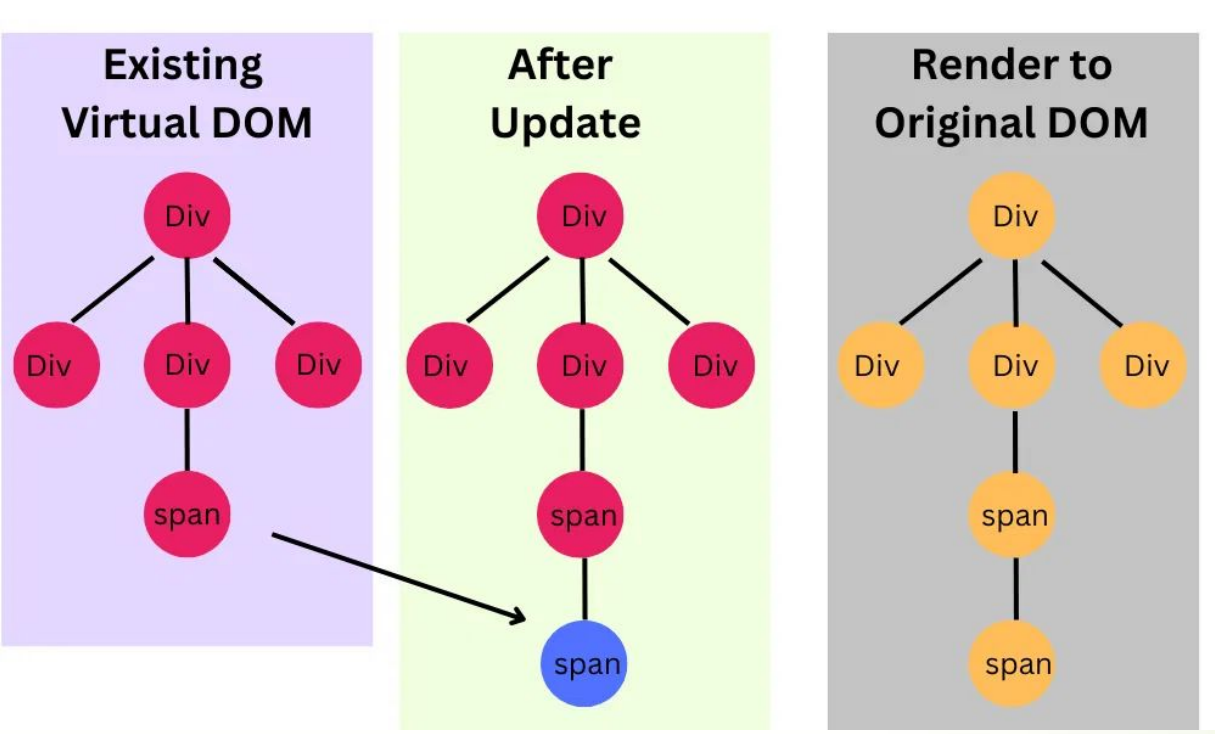
- **Tipos de Elementos Diferentes**: Se os tipos mudam (ex: para), o React reconstrói a árvore do zero
- **Tipos de Elementos Iguais**: Mantém o nó do DOM e atualiza apenas os atributos ou estilos alterados



Diffing em Ação

HEURÍSTICA PARA EFICIÊNCIA O(n)

- **A Importância das Keys**: Em listas, as keys permitem identificar quais itens mudaram, foram movidos ou removidos



JSX

- JSX = JavaScript XML
- É uma extensão de sintaxe para JavaScript que permite escrever marcação similar a HTML dentro de arquivos JavaScript
- Parece HTML, mas é JavaScript por baixo dos panos
- Transpilado para JavaScript antes da execução
- Preferido pela maioria dos desenvolvedores React
- Permite misturar marcação e lógica no mesmo lugar

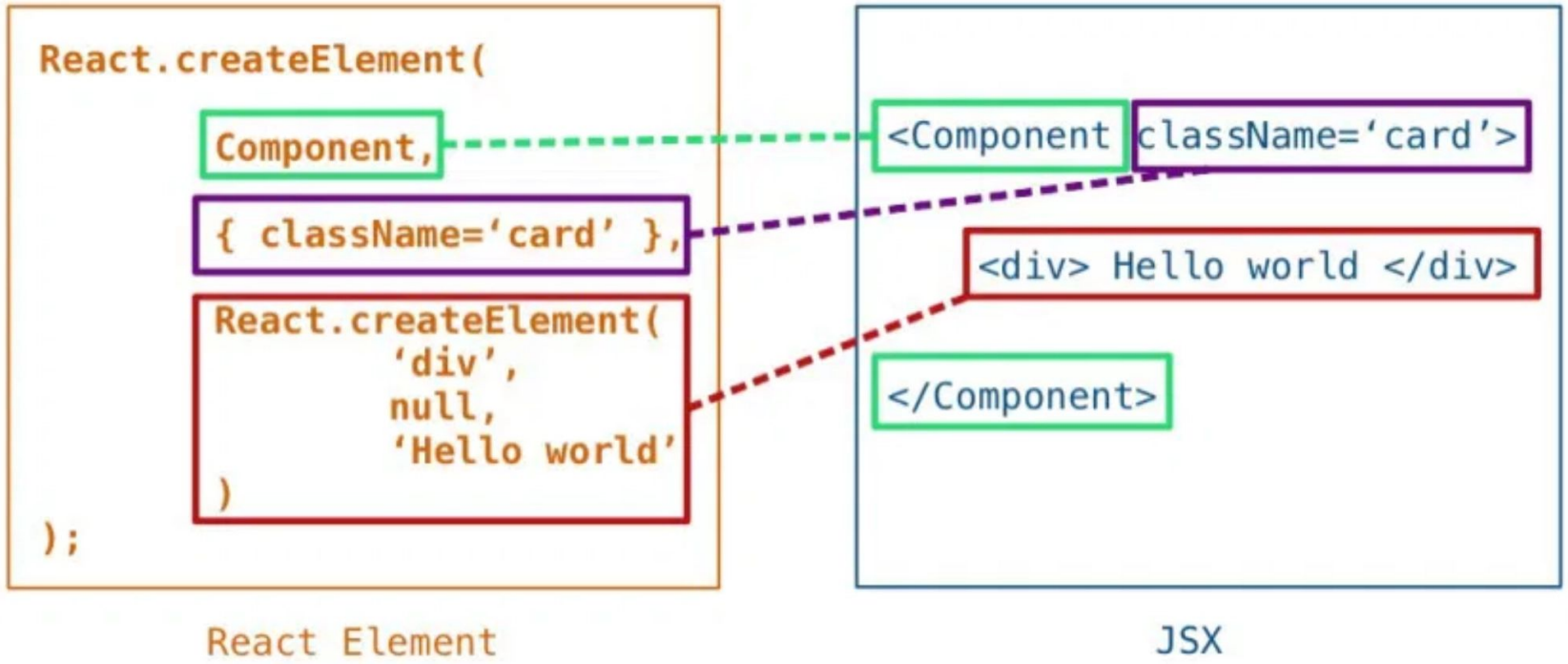
JSX

- JSX = JavaScript XML
- É uma extensão de sintaxe para JavaScript que permite escrever marcação similar a HTML dentro de arquivos JavaScript
- Parece HTML, mas é JavaScript por baixo dos panos
- Transpilado para JavaScript antes da execução
- Preferido pela maioria dos desenvolvedores React
- Permite misturar marcação e lógica no mesmo lugar

Importante: JSX e React são coisas separadas. Frequentemente são usados juntos, mas você pode usá-los independentemente.

JSX

- Exemplo com e sem JSX



JSX

- Exemplo com e sem JSX (sem utilização de função)

```
const myElement = React.createElement('h1', {}, 'Sem JSX!');

createRoot(document.getElementById('root')).render(
  myElement
 );
```

```
const myElement = <h1>I Love JSX!</h1>;

createRoot(document.getElementById('root')).render(
  myElement
 );
```

Regras do JSX

ELEMENTO RAIZ ÚNICO

- Todo componente deve retornar um único elemento raiz
- Use ou Fragment (<></>)

❌ Incorreto

```
return (
  <h1>Título</h1>
  <p>Parágrafo</p>
);
```

✅ Correto

```
return (
  <>
    <h1>Título</h1>
    <p>Parágrafo</p>
  </>
);
```

Regras do JSX

FECHAR TODAS AS TAGS

- JSX é mais rigoroso que HTML. Todas as tags devem ser explicitamente fechadas

❌ Incorreto

```
<img src="https://….">
<br>
<input type="text">
```

Regras do JSX

camelCase PARA ATRIBUTOS

- Atributos em JSX usam camelCase em vez de kebab-case
- class se torna className

❌ Incorreto

```
<div class="container"
    background-color="blue">
  <label for="name">Nome:</label>
</div>
```

✅ Correto

```
<div className="container"
    backgroundColor="blue">
  <label htmlFor="name">Nome:</label>
</div>
```

JSX com Chaves

- JSX permite usar chaves { } para incorporar expressões JavaScript diretamente na marcação, tornando o conteúdo dinâmico
- Onde usar chaves:
  - Como texto dentro de uma tag:

    <h1>{name}'s List</h1>
- Como atributos após o =:

  src={avatar}

Componente React

BLOCOS DE CONSTRUÇÃO

- **Independência**: Divida a UI em partes isoladas e reutilizáveis.
- **Funcionais**: A maneira moderna e recomendada de escrever React.
- **Props**: Argumentos que permitem passar dados entre componentes.

```
function Welcome(props) {
  return <h1>Olá, {props.name}</h1>;
}
```

Componente React

- FUNÇÃO
- Um componente React pode ser uma função JavaScript
- **Nome**: Deve começar com letra maiúscula (PascalCase)
- **Return**: Retorna marcação JSX que será renderizada
- **JSX**: Sintaxe que parece HTML mas é processada como JavaScript

Componente React

- FUNÇÃO

```
// Componente React básico
function Welcome() {
  return (
    <div className="welcome-container">
      <h1>Olá, Mundo!</h1>
      <p>Bem-vindo ao React</p>
    </div>
  );
}
```

Componente React

- FUNÇÃO

Para ser utilizada como módulo, essa função pode ser exportada

```
// Componente React básico
export default function Welcome() {
  return (
    <div className="welcome-container">
      <h1>Olá, Mundo!</h1>
      <p>Bem-vindo ao React</p>
    </div>
  );
}
```

Anatomia de um Componente React

CRIANDO O PRIMEIRO COMPONENTE

- Definir a função (nome com maiúscula)

```
function Greeting() {
```

- Adicionar marcação (return JSX)

```
return (<h1>Bem-vindo ao React!</h1>);
```

Importante: Nomes de componentes devem começar com letra maiúscula para que o React os reconheça como componentes

React em Produção

PRIMEIROS PASSOS

- Criar o aplicativo

```
npx create-react-app meu-app
```

- Criar o aplicativo com vite (mais utilizado)

```
npm create vite@latest my-react-app -- --template react
```

- Caso o vite não esteja instalado, será necessário instalar antes

```
npm install -g create-vite
```

React em Produção

UTILIZANDO O VITE

- Entrar no diretório do aplicativo

```
cd my-react-app
```

- Rodar a aplicação

```
npm install
npm run dev
```

React em Produção

UTILIZANDO O CRA

- Entrar no diretório do aplicativo

```
cd my-react-app
```

- Rodar a aplicação

```
npm start
```

React em Produção

EDITANDO O APLICATIVO

- Após iniciar o aplicativo, um servidor será criado no localhost
- Ao abrir o endereço no localhost será possível ver a aplicação
- Para editar o site, deve-se abrir a pasta do aplicativo com um editor de texto
- Os arquivos .js e .jsx criados durante o desenvolvimento serão criados na pasta /src
- Para realizar mudanças no site pode-se importá-los no arquivo App.jsx ou main.jsx

Exercício prático

- Abrir o arquivo App.jsx
- Apagar o conteúdo do arquivo e incluir o seguinte código

```
function App() {
  return (
    <div className="App">
      <h1>Hello World!</h1>
    </div>
  );
}
export default App;
```

Exercícios

1. Exercício 1: Incluindo um Componente

Crie um componente funcional chamado MeuPrimeiroComponente que renderize um elemento <h2> com o texto: "Meu primeiro componente!"

2. Exercício 2: JSX e Expressões

Crie uma variável com seu nome e a exiba dinamicamente dentro de um parágrafo <p> em um novo componente chamado PerfilUsuario.

Exercício Virtual DOM e JSX

Exercício 1: Incluindo um Componente

- Visualizando o Virtual DOM
- Objetivo: Entender como o React atualiza o DOM de forma eficiente.

1. Crie um componente React simples que exiba um contador.
2. Adicione um botão que, ao ser clicado, incremente o contador.
3. Abra as ferramentas de desenvolvedor do navegador e observe a aba "Elements" (ou "Inspetor").
4. Ao clicar no botão, note que apenas o texto do contador é atualizado no DOM real, e não o elemento inteiro.

Exercício Virtual DOM e JSX

Exercício 2: Explorando JSX

- Objetivo: Praticar a sintaxe JSX e a incorporação de expressões JavaScript.

1. Crie um componente chamado InformacoesPessoais.
2. Dentro deste componente, declare as seguintes variáveis JavaScript:

- nomeCompleto (string)
- idade (número)
- profissao (string)
- estaAtivo (booleano)

Exercício Virtual DOM e JSX

Exercício 2: Explorando JSX

- Objetivo: Praticar a sintaxe JSX e a incorporação de expressões JavaScript.

3. Utilize JSX para renderizar estas informações em elementos HTML, incorporando as variáveis usando {}.
4. Adicione uma condição para exibir a profissão apenas se estaAtivo for true.
5. Renderize o componente InformacoesPessoais no seu App principal.